

Programming 2

SEMM Primer 2026
William Wang

Outline

- Review of loops
- Review of methods/functions
- Example 1: File input/output
- Example 2: Newton Raphson
- OpenSees intro
- Walkthrough of OpenSees code

Hands on in Jupyter Lab / Colab

Programming basics - Loops

- For loop

- You know how many times to loop
- `for i in range(5):`
- `for n in [1, 2, 3, 4]:`
- `For i, letter in enumerate(['a', 'b', 'c']):`
- Can stop early if you want

- While loop

- You don't know how many times to loop
- Need to always be able to stop the loop
 - `Condition to loop (while error < tolerance:)`
 - `break`

Programming basics - Methods/functions

- Do not copy paste lines of code, put it into a method/function!
- If you use the same chunk of code multiple times, try to put it into a function
- `def increment_all(values):`
 - `answer = []`
 - `for v in values:`
 - `answer.append(v + 1)`
 - `return answer`
- `def increment_all(values):`
 - `return [i + 1 for i in values]` (w/ list comprehension)

Programming basics - Methods/functions

- Functions can return things or edit the input
- ```
def increment_all(values):
 for i in range(len(values)):
 values[i] = values[i] + 1
```
- Function parameters can have default values
  - ```
def integrate(values, dx=0.1):
```
- You can decorate function parameters for readability
 - ```
def integrate(values:"list of values", dx:"step"=0.1) ->
 "trapezoidal rule sum":
```

# Programming basics - Methods/functions

Variables inside the function are locally scoped to that function

```
A = 10
```

```
Def foo():
```

```
 A = 5
```

```
 return A
```

```
print(A) # This will print 10
```

```
print(foo()) # This will print 5
```

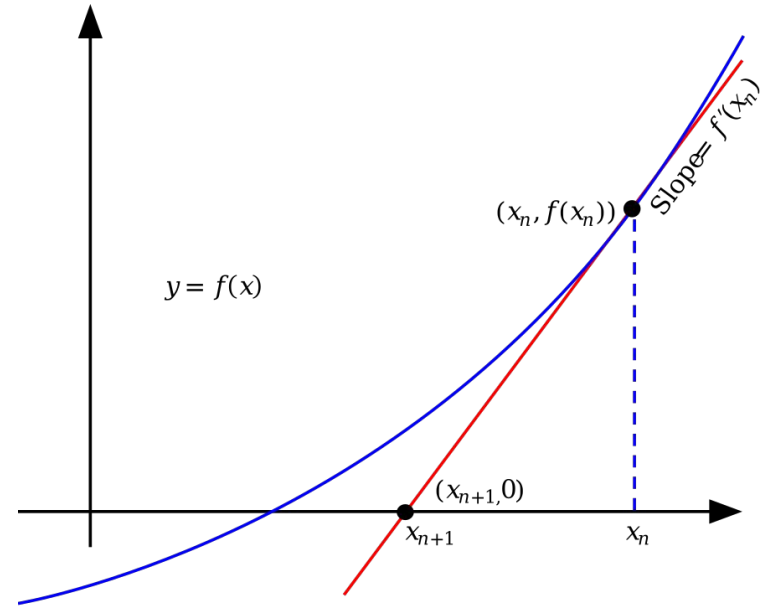
# Example 1: File input / output

- Read in ground motion data from NGA West 2
  - <https://peer.berkeley.edu/research/nga-west-2>
- Plot it
- Integrate it twice to get displacements
- Rotate the displacement orbits

## Example 2: Newton Raphson

- Solve for the root of a function
- Iterative approach
- Just needs the derivative
- Useful for vector calculations ( $\mathbf{f} = \mathbf{Ku}$ )

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$



- Need derivative to not be 0 (i.e. not singular)



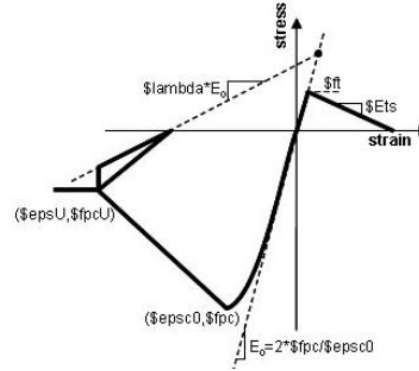
# OpenSees

- <https://opensees.berkeley.edu/>
- Structural analysis software using finite element approach
- Common in research
- More efficient, transparent, editable, adaptable than SAP2000/ETABS
  - But, much higher learning curve, easier to make mistakes
  - Need to develop own plotting and visualization to see outputs
  - Need to be consistent with units, coordinate systems, dof numbering, local degrees of freedom
  - Need to know what you're doing! Check with hand calcs! **Does it make sense?**
- Can work with it in Python (OpenSeesPy, Xara)

# OpenSees - Main workflow

## Define

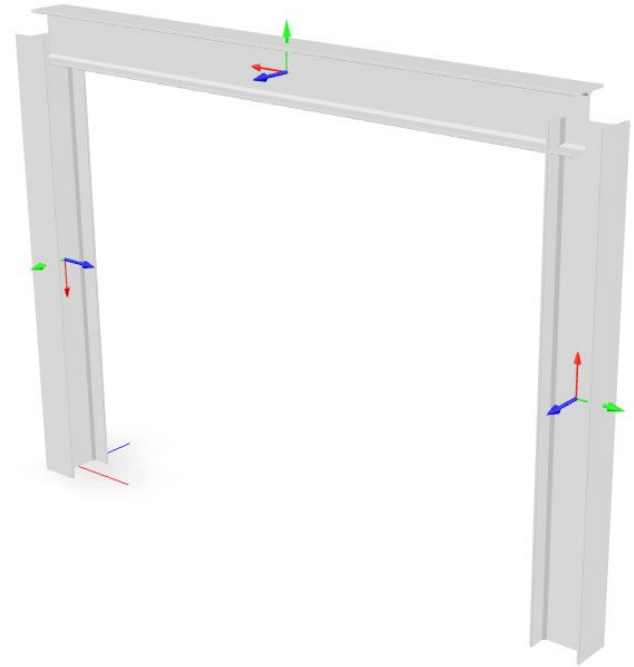
- Material responses (concrete, steel)
- Element types, sections and orientations
  - E.x. fiber sections, linear elastic prisms, zeroLength springs
- Geometry (nodes, element connections, fixed dofs)
- Loads (masses, load directions and nodes, uniform accelerations)
- Analysis (analysis settings, loop)
- Recorders (output nodal deformations, reactions, element stresses, strains)
- Post processing (read recorder outputs and make sense of it all)



[https://opensees.berkeley.edu/wiki/index.php/Concrete02\\_Material\\_-\\_Linear\\_Tension\\_Softening](https://opensees.berkeley.edu/wiki/index.php/Concrete02_Material_-_Linear_Tension_Softening)

# OpenSees - Coordinate system

- Define 2D or 3D system
  - 3 dofs in 2D
  - 6 dofs in 3D
- Orientation of sections matter (geomTransf)
- Choose your global coordinate system
  - Usually 'Y' or 'Z' axis is vertically upwards
  - Follow right-hand rule conventions
- Local coordinate system
  - Each element has local 'x' from node i to node j
  - Local 'y' and 'z' are based on the geomTransf used
  - Available output local strains are based on element
    - E.x. beam column has axial strain, 2 curvatures at each end, and torsional dof



<https://xara.so/examples/frames/frame-0059.html>

# OpenSees - Quirks

- Everything has a tag
  - Material tag, element tag, node tag, load pattern tag, time history tag....
- Element node order matters
  - Node 1 -> Node 2 is not the same as Node 2 -> Node 1
  - <https://openseesdigital.com/2022/05/08/switching-sides/>
- Recorder outputs are not labelled
  - Order of outputs may be different, usually axial, two shears, torsion, two moments
  - Need to do some detective work to properly label
- Error outputs are not obvious
  - Write own debug statements / logging errors
  - Start from simple examples and build up piece by piece

# Walkthrough 1: Uniaxial materials

<https://xara.so/examples/material/material-0002.html>

[https://opensees.berkeley.edu/wiki/index.php/UniaxialMaterial\\_Command](https://opensees.berkeley.edu/wiki/index.php/UniaxialMaterial_Command)

# Walkthrough 2: OpenSees Moment Curvature Analysis

<https://xara.so/examples/sections/fiber-0004.html>

<https://openseespydoc.readthedocs.io/en/latest/src/MomentCurvature.html>